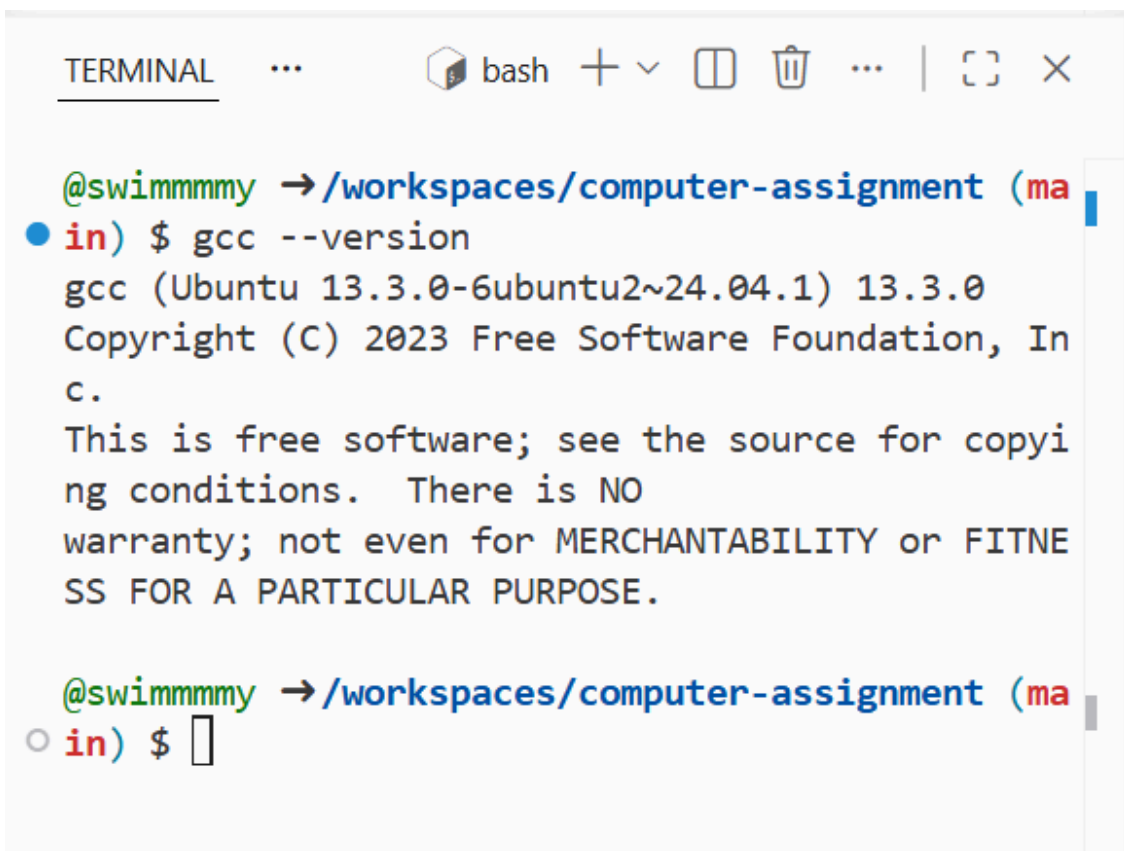


Computer Assignment

과제 1. 깃허브 코드스페이스로 리눅스 개발환경 만들기

GitHub에서 `computer-assignment` 레포지토리를 생성하고 Codespace를 실행했다.

GCC 컴파일러가 정상적으로 설치되어 있으며, C 프로그램을 컴파일할 수 있는 환경이 준비된 것을 확인하였다.



```
TERMINAL  ...  bash + ▾  [ ]  [X]  ...  |  [ ]  X

@swimmmmy → /workspaces/computer-assignment (main)
● in) $ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

@swimmmmy → /workspaces/computer-assignment (main)
○ in) $ [ ]
```

과제 2. 오버플로 예제를 언더플로로 바꿔서 해보기

char 타입은 -128부터 127까지의 값을 저장할 수 있다. 가장 작은 값인 -128에서 1을 빼면 저장 가능한 범위를 벗어나므로 127로 변경된다. 이러한 현상을 **언더플로(Underflow)**라고 한다.

추가설명

char 가 저장할 수 있는 범위는 **-128 ~ 127**이다.

현재 값이 **-128**일 때 여기서 **1을 더 빼면 -129**가 되어야 하지만, char 는 **-129를 저장할 수 없기 때문에 127로 돌아간다.**

char 는 **8비트(1바이트)**로 값을 저장한다.

즉, 저장할 수 있는 경우의 수는 **256개(2^8)**뿐이다.

부호 있는 char 의 범위는 다음과 같다.

- -128 ~ -1
- 0 ~ 127

즉, 가장 작은 값은 **-128**, 가장 큰 값은 **127**이다.

이를 원처럼 생각하면 이해하기 쉽다.

```
... → 125 → 126 → 127
                        ↓
-128 ← -127 ← -126 ← ...
```

- **127에서 1을 더하면** 저장 범위를 넘어 **-128**로 돌아간다. (오버플로)
- **-128에서 1을 빼면** 저장 범위를 넘어 **127**로 돌아간다. (언더플로)

즉, 컴퓨터는 **8비트 안에서만 계산하기** 때문에 저장 가능한 범위를 벗어나면 값이 반대쪽 끝으로 이동한다. 이러한 현상을 **언더플로(Underflow)**라고 한다.

코드 (char_underflow.c)

```
#include <stdio.h>
#include <limits.h>

int main() {
    char value = CHAR_MIN;

    printf("Original value: %d\n", value);
```

```

    value = value - 1;

    printf("Value after subtracting 1: %d\n", value);

    return 0;
}

```

중요한 코드 설명

1. 최솟값으로 초기화

```
char value = CHAR_MIN;
```

- `CHAR_MIN` 은 `char` 형이 저장할 수 있는 **최솟값(-128)**을 의미한다.
- 변수 `value` 를 `-128` 로 초기화한다.

2. 언더플로 발생

```
value = value - 1;
```

- `value` 에서 1을 뺀다.
- 원래 결과는 `-129` 가 되어야 하지만, `char` 형은 **-128 ~ 127**까지만 저장할 수 있다.
- 따라서 저장 범위를 벗어나 **127로 변경되며 언더플로가 발생한다.**

3. 결과 출력

```

printf("Original value: %d\n", value);
printf("Value after subtracting 1: %d\n", value);

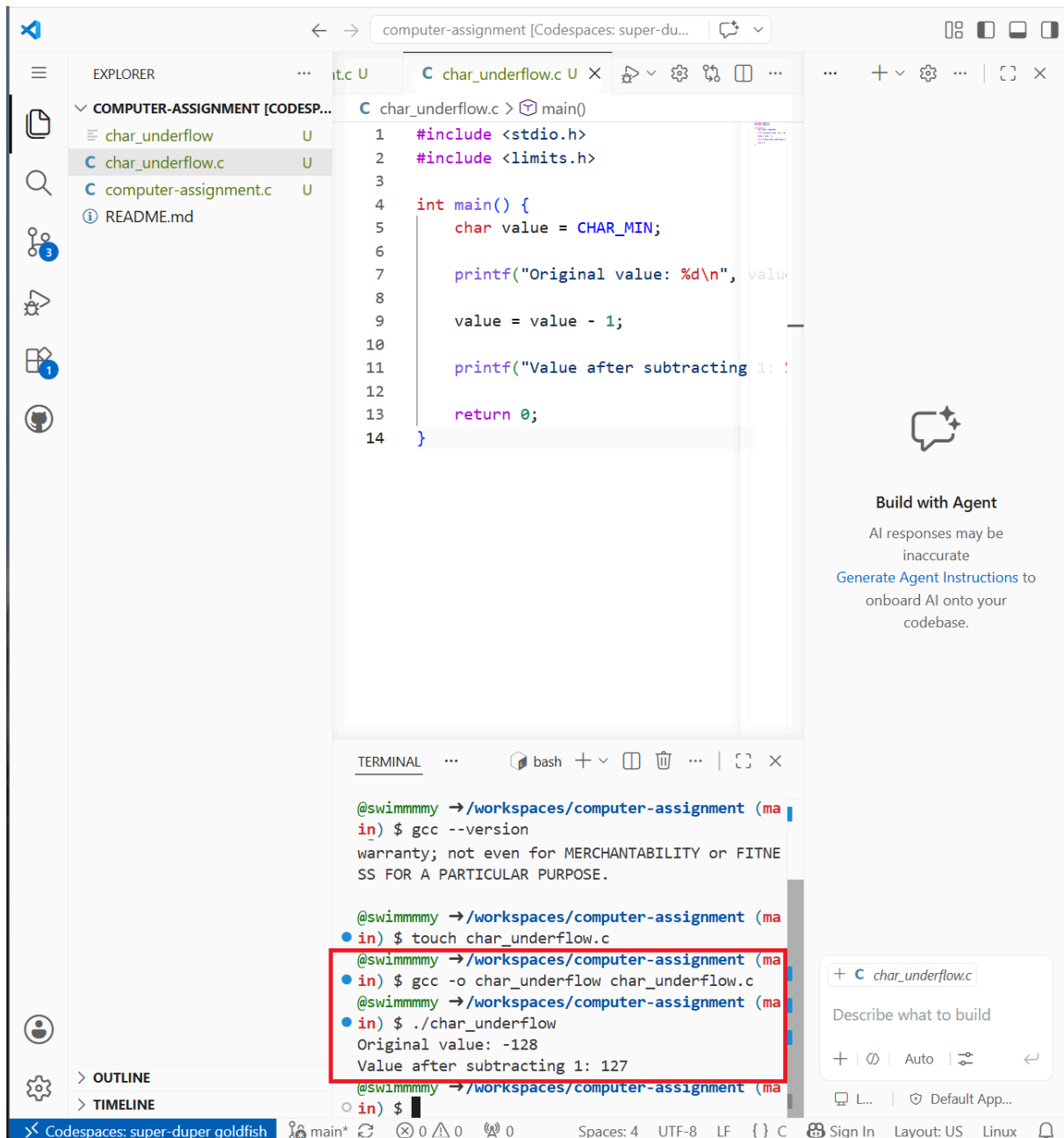
```

- 첫 번째 `printf()` 는 연산 전 값(-128)을 출력한다.
- 두 번째 `printf()` 는 언더플로가 발생한 후의 값(127)을 출력한다.

실행 결과

`char` 는 8비트 부호 있는 정수(-128 ~ 127)이다.

-128에서 1을 빼면 비트가 `10000000` → `01111111`로 바뀌어 127이 된다.



과제 3. 비트 연산 프로그램 바꿔보기

설명

기존 코드에는 **is_bit_set** 함수와 **set_bit** 함수만 구현되어 있었으며, 이번 과제에서는 **clear_bit** 함수를 추가로 구현하였다.

- **is_bit_set** : 특정 위치의 비트가 1(켜져 있는 상태)인지 확인하는 함수이다.
- **set_bit** : 특정 위치의 비트를 1로 설정(켜기)하는 함수이다.
- **clear_bit** : 특정 위치의 비트를 0으로 변경(끄기)하는 함수로, 이번 과제에서 직접 추가하였다.

set_bit 함수는 OR(**|**) 연산을 사용하여 원하는 위치의 비트를 1로 설정한다. 예를 들어 **00000100(4)** 에서 3번째 비트를 켜면 **00001100(12)** 가 된다.

코드 (bit_operation.c)

```
#include <stdio.h>

int is_bit_set(unsigned char value, int position) {
    return (value & (1 << position)) != 0;
}

unsigned char set_bit(unsigned char value, int position) {
    return value | (1 << position);
}

unsigned char clear_bit(unsigned char value, int position)
{
    return value & ~(1 << position);
}

int main() {
    unsigned char value = 0b00001000;

    if (is_bit_set(value, 3)) printf("3rd bit is set!\n");
    else printf("3rd bit is not set!\n");

    value = set_bit(value, 2);
    printf("Value after setting 2nd bit: %d\n", value);

    value = clear_bit(value, 3);
    printf("Value after clearing 3rd bit: %d\n", value);
}
```

```
    return 0;
}
```

중요한 코드 설명

1. 특정 비트가 켜져 있는지 확인

```
return (value & (1 << position)) != 0;
```

- `1 << position` 으로 확인할 비트 위치를 만든다.
- `&` (AND) 연산을 이용하여 해당 비트가 1인지 확인한다.
- 결과가 0이 아니면 해당 비트가 켜져 있다는 의미이다.

2. 특정 비트 켜기

```
return value | (1 << position);
```

- `1 << position` 으로 원하는 위치의 비트를 만든다.
- `|` (OR) 연산을 수행하여 해당 비트를 1로 설정한다.

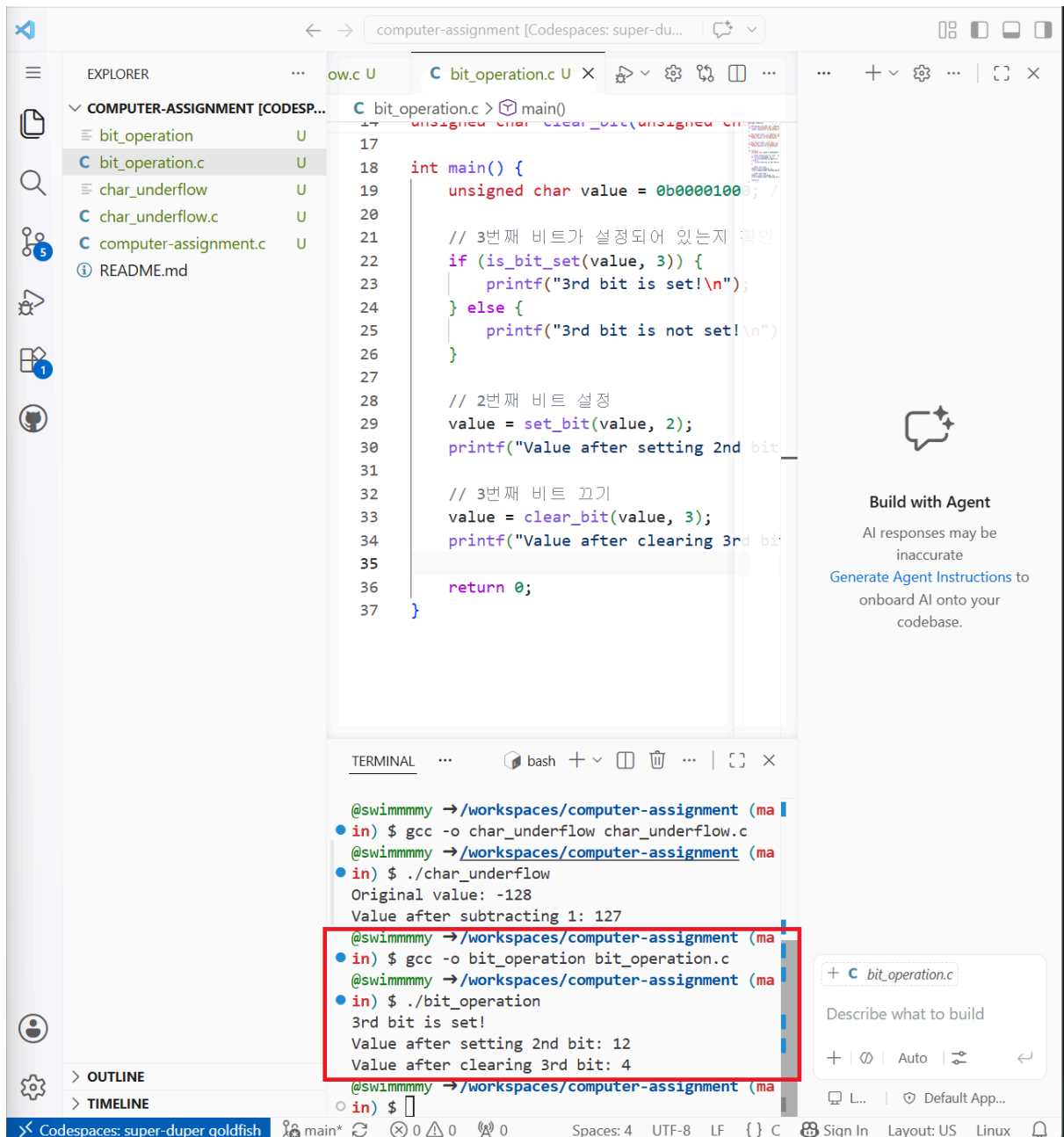
3. 추가한 clear_bit 함수

```
return value & ~(1 << position);
```

- `1 << position` 으로 원하는 위치의 비트를 만든다.
- `~` (NOT) 연산으로 해당 위치만 0이 되도록 반전시킨다.
- `&` (AND) 연산을 수행하여 원하는 위치의 비트만 0으로 바꾸고 나머지 비트는 그대로 유지한다.

실행 결과

프로그램을 실행한 결과, `is_bit_set` 함수는 특정 비트가 설정되어 있는지 정상적으로 확인하였고, `set_bit` 함수는 원하는 비트를 1로 설정하였다. 또한 직접 추가한 `clear_bit` 함수는 해당 비트를 0으로 변경하는 것을 확인할 수 있었다.



추가. 사용자 입력 받도록 수정

기존에는 값이 코드에 고정되어 있었는데, 사용자가 직접 값과 비트 위치를 입력하도록 수정하였다. 입력값을 바꿔가며 비트 연산 결과를 확인할 수 있었다.

```

int main() {
    unsigned char value;
    int input_value, position;

```

```

printf("값을 입력하세요 (정수): ");
scanf("%d", &input_value);
value = (unsigned char)input_value;

printf("비트 위치를 입력하세요 (0-7): ");
scanf("%d", &position);

printf("현재 값: %d\n", value);

if (is_bit_set(value, position)) {
    printf("%d번째 비트가 설정되어 있습니다!\n", position);
} else {
    printf("%d번째 비트가 설정되어 있지 않습니다!\n", position);
}

value = set_bit(value, position);
printf("%d번째 비트 설정 후 값: %d\n", position, value);

value = clear_bit(value, position);
printf("%d번째 비트 해제 후 값: %d\n", position, value);

return 0;
}

```

중요한 코드 설명 (입력 버전)

1. 사용자에게 값 입력받기

```

printf("값을 입력하세요 (정수): ");
scanf("%d", &input_value);
value = (unsigned char)input_value;

```

- 사용자가 정수를 입력하면 `scanf()` 를 통해 입력받는다.
- 입력받은 값을 `unsigned char` 형으로 변환하여 `value` 에 저장한다.

2. 비트 위치 입력받기


```
printf("비트 위치를 입력하세요 (0-7): ");  
scanf("%d", &position);
```

- 사용자가 확인하거나 변경할 비트의 위치(0~7)를 입력받는다.

3. 입력한 비트 확인

```
if (is_bit_set(value, position))
```

- `is_bit_set` 함수를 호출하여 입력한 위치의 비트가 1인지 확인한다.
- 결과에 따라 비트가 설정되어 있는지 출력한다.

4. 비트 설정 및 해제

```
value = set_bit(value, position);
```

- 입력한 위치의 비트를 1로 설정한다.

```
value = clear_bit(value, position);
```

- 입력한 위치의 비트를 0으로 변경한다.

실행 결과 (입력: 값=4, 위치=3)

입력값을 4, 비트 위치를 3으로 입력한 결과, 3번째 비트가 설정되고 다시 해제되는 것을 확인할 수 있었다.

```
@swimmmmy → /workspaces/computer-assignment
● (main) $ ./bit_operation
값을 입력하세요 (정수): 4
비트 위치를 입력하세요 (0-7): 3
현재 값: 4
3번째 비트가 설정되어 있지 않습니다!
3번째 비트 설정 후 값: 12
3번째 비트 해제 후 값: 4
@swimmmmy → /workspaces/computer-assignment
○ (main) $
```

과제 4. C언어가 기계어가 되는 과정

`hello_world.c` 를 단계별로 컴파일하여 C 언어가 실행 파일이 되는 과정을 확인하였다.

소스 코드 (hello_world.c)

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

컴파일 파이프라인

단계	명령어	출력 파일	설명
전처리	<code>gcc -E hello_world.c -o hello_world.i</code>	hello_world.i	헤더 파일 포함, 매크로 치환
컴파일	<code>gcc -S hello_world.i -o hello_world.s</code>	hello_world.s	어셈블리 코드 생성
어셈블	<code>gcc -c hello_world.s -o hello_world.o</code>	hello_world.o	오브젝트 파일 생성

단계	명령어	출력 파일	설명
링킹	<code>gcc hello_world.o -o hello_world</code>	hello_world	실행 파일 생성

파일 크기 비교

컴파일 단계가 진행되면서 생성되는 파일의 종류와 크기가 달라지는 것을 확인하였다.

파일	크기	설명
hello_world.c	79 bytes	소스 코드
hello_world.i	21KB	전처리 후 (헤더 포함)
hello_world.s	674 bytes	어셈블리 코드
hello_world.o	1.5KB	오브젝트 파일
hello_world	16KB	실행 파일

실행 결과

컴파일을 진행하면서 전처리, 컴파일, 어셈블, 링킹 단계를 거쳐 최종 실행 파일이 생성되는 과정을 확인할 수 있었으며, 단계별로 생성되는 파일의 종류와 크기가 달라지는 것도 확인하였다.

